2026 INTERNATIONAL FINTECH COMPETITION / TOPIC C

StudentBridge AI

Full project guide and implementation manual

A complete six-country student-finance project, including verified sources, collection strategy, technical features, launch steps, competition requirements and final team tasks.

StudentBridge AI · China · United Kingdom · Japan · South Korea · Australia · Bangladesh

Prepared 15 October 2026 · Demonstration system · Real references with conditional user-input quotes

StudentBridge AI — MySQL Edition

2026 FinTechathon International Track / Topic C: Cross-border & Student Finance Assistant.

A working six-country student finance demonstrator for **China, United Kingdom, Japan, South Korea, Australia and Bangladesh**, covering all 30 possible **directional** country pairs. Each corridor can be selected; the application does **not** claim every bank transfer is legally or commercially available.

Working features

- EduRoute: compare user-entered provider quotes with deterministic fees, deadlines, affordability and eligibility assertions; no invented bank offers.
- Financial Digital Twin: stress-test exchange rates, added fees and living expenses without presenting a forecast as a fact.
- Explainable Decision Passport: every option's calculation, rejection reasons, source labels and decision fingerprint.
- PDF tuition invoice analysis: candidate amounts/dates; user must confirm all fields.
- Official source catalog and reference FX API: rates are **reference rates**, not customer-specific remittance prices.
- Six-country educational credit guidance, source-backed research tables, and security cautions.
- MySQL-based sandbox payment transaction records, idempotency protection, atomic HMAC-linked audit events; **no real money is moved**.
- Optional Gemini for rephrasing grounded results, not approving payments or generating fees.

Fast start — macOS / Linux

Requirements: Python 3.10+, **MySQL Server 8.0+**, and Internet for live reference FX. Use the detailed MySQL setup guide if this is your first time.

1. Create a MySQL database called `studentbridge_ai` and a limited-privilege MySQL user (SQL in `docs/MYSQL_SETUP.md`). You can use MySQL Workbench or optional `docker-compose.yml`.
2. From the project root, configure the environment and install packages:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
```

1. Edit `.env`: set your real database password in `DATABASE_URL` and replace `APP_SECRET` with a strong secret.
2. Initialize MySQL, then start the app:

```
python scripts/setup_mysql.py
python scripts/check_mysql.py
python run.py
```

1. Open **http://127.0.0.1:8000**. `/api/health` should show `"database_engine": "mysql"`.

For Windows PowerShell, activate with `.venv\Scripts\Activate.ps1` and copy `.env.example` to `.env` in File Explorer or with `Copy-Item`.

Database structure (10 tables)

Application: `plans`, `transactions`, `audit`, `audit_chain_state`.

Research: `countries`, `sources`, `financial_benchmarks`, `tuition_benchmarks`, `fx_rates`, `payment_quotes`.

The corrected 15 October 2026 research seed is imported from `data/research/corrected/*.csv`. Missing rates/quotes remain NULL, not zero. No verified bank quotes are present. The original uploaded reference material is preserved in `data/research/uploaded_originals/` (except old SQLite binary databases, removed from this MySQL edition).

Useful files:

File	Purpose
`app/main.py`	FastAPI endpoints and agent interaction
`app/database.py`	MySQL tables, transactions, audit-chain locking and data importer
`app/logic.py`	Calculation and Digital Twin engine
`app/static/`, `app/templates/`	Working responsive interface
`scripts/mysql_schema.sql`	Pure MySQL 8 DDL
`scripts/mysql_research_seed.sql`	Pure MySQL research inserts
`scripts/setup_mysql.py`	Safe MySQL initializer and CSV importer
`scripts/check_mysql.py`	MySQL connectivity/integrity check
`scripts/seed_sandbox.py`	Generate new local MySQL example evidence
`tests/`	API, logic, financial data and schema tests
`docs/MYSQL_SETUP.md`	Full beginner-friendly MySQL guide
`docs/SECURITY_SELF_ASSESSMENT.md`	Implemented controls and security limitations

Test the project

```
python -m pytest -q
```

Test isolation: automated tests use an ephemeral SQLite database to validate application behavior without writing to your MySQL data. That is a test implementation detail, not the runtime database. The application requires a real MySQL connection outside test mode. Because the project was built in an environment without a MySQL server and without access to download a MySQL driver, live server integration must be checked on your own MySQL environment using `python scripts/check_mysql.py`, `MYSQL_SMOKE_TEST=YES python scripts/smoke_test_mysql.py` (dedicated test DB only) and a browser walkthrough.

Competition submission mapping

The uploaded official competition rules request: technical design, a presentation (max 10 min), source code with GitHub and deployment instructions/test cases, security self-assessment, and sandbox execution evidence. They also list an optional demo video (max 5 min) while requesting demo video as part of non-on-chain execution evidence; record a short video to cover that requirement.

- Technical design: `docs/SYSTEM_DESIGN.md` and `docs/TECHNICAL_DOCUMENTATION.pdf`.
- Security report: `docs/SECURITY_SELF_ASSESSMENT.md` / `.pdf`.
- Presentation: `docs/submission/StudentBridge_AI_Presentation.pptx`.
- Demo instructions: `docs/DEMO_VIDEO_SCRIPT.md`.
- Evidence: run `scripts/seed_sandbox.py` to produce NEW MySQL sandbox logs.
- Publish your own reviewed/legally compliant GitHub repository before submission.

Serious limitations

This is a *competition sandbox*, **not a public banking product**. It does not authenticate students, isolate records by user, perform identity verification, implement actual KYC/AML controls or independently validate provider quotes. Do not expose it publicly with real user financial data. No bank or licensed payment processor is connected. Only students' hypothetical test quotes should be used in the demo. Check data-source licenses and independently review project originality before submission.

MySQL 8 Setup for StudentBridge AI (Mac / Windows / Linux)

The application now uses **MySQL 8.0+** for **all persistent application and research records**. It does not use an on-disk SQLite database in normal operation. SQLite is used only by disposable automated unit tests, never for deployment. Official sources and design catalog remain in read-only JSON/CSV files as an offline reference; the corrected six-country financial research seed is also imported to MySQL.

Step 1: Install MySQL and create the database

Use MySQL Community Server 8.x / MySQL Workbench, a MySQL 8.4 Docker container, or managed MySQL (Railway/Aiven/etc.). Keep the database private; never expose port 3306 to the public Internet.

Run in MySQL Workbench with an administrator account (replace both placeholders):

```
CREATE DATABASE IF NOT EXISTS studentbridge_ai CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
CREATE USER IF NOT EXISTS 'studentbridge_app'@'localhost' IDENTIFIED BY
'REPLACE_WITH_A_LONG_PRIVATE_PASSWORD';
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX, REFERENCES
ON studentbridge_ai.* TO 'studentbridge_app'@'localhost';
```

For a Docker-based local DB, you can instead set `MYSQL_ROOT_PASSWORD` and `MYSQL_APP_PASSWORD` in your private `.env`, run `docker compose up -d mysql`, and use the automatically created user and database. NOTE: `.env.example` contains the Python DATABASE_URL settings and should be copied to `.env` first.

If the application runs on a different server, its MySQL user host must match the deployed environment. Prefer the host's managed MySQL connection configuration and TLS rather than allowing anonymous/public access.

Step 2: Install Python requirements

From the project root:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Windows PowerShell activation is `.venv\Scripts\Activate.ps1`.

Step 3: Configure .env

```
cp .env.example .env
```

Change the URL to the MySQL host, port, user, password and database name. Example:

```
DATABASE_URL=mysql+pymysql://studentbridge_app:YOUR_URL_ENCODED_PASSWORD@127.0.0.1:3306/
studentbridge_ai?charset=utf8mb4
APP_ENV=development
APP_SECRET=CHOOSE_A_RANDOM_32_CHARACTER_OR_LONGER_SECRET
```

If your password contains `@`, `:`, `/`, `#`, `%`, or spaces, URL-encode those characters in DATABASE_URL. Never commit `.env` or credentials to GitHub. Use a unique MySQL login with limited privileges and set secure secret values for deployment. The app loads `.env` automatically using python-dotenv.

Step 4: Initialize and import verified research rows

```
python scripts/setup_mysql.py
```

This creates 10 tables (4 operational/audit and 6 research tables) and imports the corrected CSV records without filling missing payment quotes. It uses an upsert for existing research IDs and never deletes saved plans or transactions. A *fresh* app startup also imports the research CSVs automatically when the country table is empty.

Alternative manual SQL import in MySQL Workbench: run `scripts/mysql_schema.sql` followed by `scripts/mysql_research_seed.sql`. The SQL seed is **INSERT IGNORE**, so it preserves existing rows; use the Python importer to update research rows by ID.

Step 5: Run and verify

```
python scripts/check_mysql.py
python run.py
```

Visit `http://127.0.0.1:8000`. Visit `http://127.0.0.1:8000/api/health` to confirm the `database_engine` field is `mysql`.

For an actual **write/read MySQL integration check** (use a disposable MySQL database):

```
MYSQL_SMOKE_TEST=YES python scripts/smoke_test_mysql.py
```

This creates one clearly labeled synthetic plan and simulated transaction and tests replay protection. Never run against production.

For full sandbox evidence (non-production test DB only), run:

```
python scripts/seed_sandbox.py
```

This creates an example `evidence/mysql_sandbox_example.json` from obviously hypothetical student/payment inputs. It is NOT a real provider quote and does NOT move money.

Step 6: Tests and deployment

```
python -m pytest -q
```

Unit and API tests deliberately run with a new disposable test-only SQLite database, so they **do not prove a real MySQL service is available**. Use `python scripts/check_mysql.py` and a real web-app walkthrough against a separate MySQL test database for MySQL integration testing. Never use production data in integration tests.

For deployment use a service that can host Python FastAPI and a managed MySQL 8 database, with a private connection and automatic backups. **Netlify cannot run the Python backend and MySQL itself**; hosting the current app (HTML served directly by FastAPI) as a single backend app is simpler than splitting the frontend. This prototype has NO user login/role isolation and MUST NOT be publicly deployed with real financial or personally identifying data. Add authentication, session isolation, TLS, rate limits, and privacy controls before real users.

Known limitations

- No provider-specific verified real payment quotes for arbitrary pairs; user-entered quote claims are NOT independently verified.
- No live bank integration; the payment ledger is sandbox only.

- Country-specific regulations are informational, not official compliance verification.
- Existing generated historical evidence from the SQLite version is labeled legacy and is NOT evidence of a new MySQL run.
- Financial data provenance, collection dates and NULL missing values are preserved in imported research tables.

Real Financial Data Collection - Exact Instructions

Scope and data status

Support country selection for China (CNY), United Kingdom (GBP), Japan (JPY), South Korea (KRW), Australia (AUD) and Bangladesh (BDT). There are 30 ordered directions, not 30 proven bank routes. All data must include source URLs, observation/effective dates and `source_type`. Missing data stays missing. Real student financial information requires consent, minimum necessary fields and secure treatment; none is bundled.

1. Live reference FX (six currencies)

Source: https://frankfurter.dev/ (v2). API example: `GET https://api.frankfurter.dev/v2/rate/JPY/GBP` returns a numerical rate and publication date, when available. The backend fetches only on demand; if internet is unreachable, it returns `available=false` with no invented rate.

Fields: `base_currency, quote_currency, published_rate, observation_date, retrieved_at, source_url, rate_type=market_reference`.

Never: call it a provider sell rate or guarantee that customers can transfer at that price.

2. Real 2026 university tuition

Sources:

- Australia (specific numeric data): https://students.unimelb.edu.au/__data/assets/pdf_file/0020/5423033/2026-International-Course-Tuition-Fees-OF_20260507.pdf
- United Kingdom: https://www.manchester.ac.uk/study/masters/courses/list//13005/ma-international-education/overview/ and https://www.ucl.ac.uk/prospective-students/graduate/taught-degrees/management-complex-projects-msc
- China: https://masters.sem.tsinghua.edu.cn/en/info/1007/2103.htm
- Japan: https://ipads.a.u-tokyo.ac.jp/admissions/cost-scholarship/ and https://www.s.u-tokyo.ac.jp/en/admission/research-students.html
- South Korea: https://ee.kaist.ac.kr/en/admission-08/
- Bangladesh: https://www.bracu.ac.bd/admissions/tuition-and-fees (per-credit published data; individual fee status matters).

Fields: `institution, country, programme, year, student_fee_status, tuition_amount, currency, payment_period, campus, source_url, updated_at, exact_invoice_confirmed`.

University-published annual/cost-per-EFTSL data must never be automatically confused with the student's actual payable invoice. The catalog now includes selected 2026 official published fee examples for all six countries; it always shows period, source and any student-status restrictions.

3. Bank and cross-border provider offers

Source: actual bank/provider customer quotation applicable to **the exact two countries and currency direction**. Ask the bank for documented tuition-purpose eligibility, payment method, signed/net beneficiary amount, quoted rate, flat fees, percentage fees, intermediary fees, cut-off time, expected settlement, quote expiry and beneficiary validation rules. A general price page is *not* a customer-specific quote.

Fields: `provider, source_country, destination_country, corridor_currency_pair, declared_eligibility, quote_rate, fixed_fee, percent_fee, correspondent_fees, beneficiary_deductions, expected_eta, quote_timestamp, expiry, reference_url_or_document_id, verification_status`.

If missing: `quote_unavailable`. Do not generate fictional "real" bank offers. Some information may require a licensed API, bank relationship, or logged-in customer account.

4. Regulatory references

Start from the official national regulators: China SAFE (`safe.gov.cn/en`), UK FCA (`fca.org.uk`), Japan FSA (`fsa.go.jp/en`), Korea FSC (`fsc.go.kr/eng/index`), Australia AUSTRAC (`austrac.gov.au`), Bangladesh Bank (`bb.org.bd`). These are *research starting points* rather than machine-readable verified eligibility for each route.

Fields: `origin, destination, type, relevant_entity, law_or_rule, allowed_conditions, document_checklist, applicable_thresholds, effective_date, source_url, reviewed_by, legal_review_status`.

The current system cannot assert that a cross-border transfer is legally compliant. Do not replace proper KYC/AML with a source URL.

5. Living costs and formal visa funds

Real examples: UK Student visa London GBP 1,529 per month / outside London GBP 1,171 per month up to 9 months, with applicable exceptions, from https://www.gov.uk/student-visa/money. JASSO Student Guide 2025-26: 2023 privately financed international student living costs JPY 105,000 monthly national average; source https://www.studyinjapan.go.jp/en/assets/pdf/sgtj_2025_english.pdf.

Fields: `country, city, amount, currency, month, category, scope, observation_year, publication_date, source_url, is_visa_rule`. Do not present visa bank-balance requirements as a guarantee of actual living affordability.

6. Credit readiness

Official/principal examples: UK MoneyHelper (`moneyhelper.org.uk`), Australian Government Moneysmart (`moneysmart.gov.au`), Japan CIC (`cic.co.jp/en/credit-report/`). Add China, South Korea and Bangladesh

details only from verified local institutions. Store `country, consumer_action, document_requirement, eligibility_caveat, official_url, effective_date`.

Credit score prediction is out of scope. Do not invent students' private credit profiles.

7. Student-provided information

The user can enter destination university, confirmed tuition amount and deadline, scholarship already awarded, actual *self-reported* available money, protected reserve and living-expense estimates. The project does not request passwords, card PIN, OTP, government identifier, passport scan or real bank credentials. The sandbox DB must not be treated as secure storage for sensitive real data.

8. Practical collection order

Day 1: load FX API reference and 6-country catalog; collect a narrow set of university fees with original links. **Day 2:** enter bank/provider quotes for actual supported routes, explicitly recording gaps and confirming expiry. **Day 3:** map tuition document conditions and visa/living references. **Day 4:** review student credit guidance and build additional institution coverage. **Day 5:** revalidate source timestamps and run financial/error tests.

9. Provenance checklist

For each record ask: Is this a *primary* source? Is it applicable to 2026 and the intended customer? Is the value a price, an estimate, a statutory requirement, or a quote? Does it specify which way the currency converts? Is it available for the requested pair? Has it expired? Is any charge missing? Are there redistribution/licensing restrictions? If any answer is unknown, display a verification-needed label.

Competition Rules Compliance Matrix (2026 FinTechathon International Track)

This project follows the user's uploaded **PBCSF Appendix 2 - Problem Description and Competition Rules for International Track** (4 pages), included at `docs/Competition_Rules.pdf`.

Topic C

Official topic: "Cross-border & Student Finance Assistant" - remittances, tuition payments, credit building and multi-currency management. StudentBridge AI supports those use cases as follows: cross-border tuition planning with a quote-aware optimized ranking; multiple currencies and historical reference FX API; personal financial digital twin; credit-readiness education; secure sandbox execution.

Judging weights

- **Task completion 40%:** end-to-end user inputs -> decision plan -> risk check -> user approval -> test transaction -> receipt -> audit event.
- **Security & compliance 30%:** controls and explicitly documented limitations. Does not claim KYC/AML screening or regulatory certification; blocking unsafe inputs and approval checks can be demonstrated.
- **Innovation & interaction 30%:** deadline-constrained optimizer, reserve-aware Digital Twin, source-linked Decision Passport, six-country choice, responsive dashboard and grounded copilot.

Submission materials required (page 3)

1. Technical documentation - provided as system design `.md` and `.pdf`.
2. Presentation - `docs/submission/StudentBridge_AI_Presentation.pptx`. Keep spoken presentation under 10 minutes.
3. Demo video - optional standalone demo, max 5 minutes; not recorded in package.
4. Source code, README, repository link, deployment directions, test cases - all provided locally except live GitHub link. **Publish to GitHub yourself**.
5. Security self-assessment - in PDF and Markdown, with permission tiers and risks.
6. Execution evidence for non-on-chain topic - sample simulated-account ledger and logs. A real **screen demonstration video still needs recording and submission**.

Administrative requirements (page 3-4)

- Teams of 2-5 students; 60% or more non-Chinese nationals for the International Track, and eligible international institutions.
- English for presentations, submissions and reviews.
- October 20, 2026 23:59 Beijing time (October 20, 2026 21:59 Bangladesh time) deadline.
- Only one team per participant; independent original work, properly licensed sources, and no plagiarism.
- Finalists: at least one member must attend in person or risk forfeiture of awards.

The submission package **is not automatically submitted** and does not imply the student's eligibility or source-license review has been completed.

Final tasks the team must complete before submitting

1. **Publish the GitHub repository.** The competition demands a GitHub repository link. Ensure your own team understands, reviews and independently develops the entry under the contest's originality rules. Do not commit API secrets, real invoice personal data, or `.env`.
2. **Provide real bank quote evidence for your demonstration pair.** Use the exact provider, customer type, sending country, destination country, payment purpose, quote timestamp, fee and expiry. The package does not contain any live verified offer; the example evidence uses synthetic quote entries that are transparently labeled.

3. **Verify every country rule for the claimed functionality.** Regulator home pages are not enough for legal eligibility. Get bank/official source review before treating real provider corridors as permitted.
4. **Run MySQL setup (`docs/MYSQL_SETUP.md`) and the software end-to-end on your own laptop.** Check `http://127.0.0.1:8000`, fetch reference FX, enter real program details, upload a permissioned invoice PDF and confirm the plan.
5. **Record the required real demonstration.** Use `docs/DEMO_VIDEO_SCRIPT.md`; show actual sandbox logs while capturing the live application. Keep the optional standalone demo video at or under 5 minutes.
6. **Review security report and replace demo secret** if running beyond private localhost. Never deploy the current single-user local MVP as a financial product without proper authentication and compliance review.
7. **Perform original-team review, license check and test rerun.** Attach quantitative performance screenshots or a test-results report; do not fabricate scores.
8. **Keep presentation under 10 minutes.** Source deck has 10 slides. Practice and tighten if needed.
9. **Submit before the official deadline.** Deadline Oct 20 2026 23:59 Beijing time, equivalent to Oct 20 21:59 Bangladesh time.